Detecting AI Training in Datacentre Power

Tom Kimpson [1] **Mauricio Baker** [2] **Emlyn Graham** [3]

Abstract

AI governance agreements that restrict large model training runs require methods to detect them on a monitored computing cluster, without trusting the operator. We develop a method to detect synchronous model training in facility-level power by Viterbi tracking of its wandering iteration cadence. On simulated datacentre power, when the operator makes no attempt to hide, the Viterbi tracker detects at least 96% of training runs against a declared inference workload, at a 1% false-alarm rate and within 0.04 of the Neyman–Pearson optimum, given a meter that samples at 2 Hz or faster. Of eight strategies an operator might use to hide, only extreme variation of the work per iteration evades the tracker. It hides training in up to 73% of five-minute observations, but over a day at least 87% of such runs are still caught. This strategy costs no throughput, but in small-scale training the operator needs 93–267% more compute to reach the same loss. A modest passive meter is sufficient to make hidden training both detectable and expensive, which makes power metering a promising part of compute verification that does not depend on trusting the operator.

1. Introduction

AI governance agreements will require regulating training runs (Shavit, 2023; Sastry et al., 2024; Baker et al., 2025; Heim & Koessler, 2024). This will require an external verifier to identify when training is occurring on a monitored cluster, against an operator who is trying to hide their training. There are multiple channels that could provide the verifier with diagnostic information to identify hidden training. These include on-chip mechanisms that log activities for later analysis, network taps that monitor data exchanged between chips, or human-centred approaches such as whistleblower programs or interviews of operator personnel. However each of these channels requires trusting either hardware that is under the control of the operator, or the operator personnel themselves. Conversely, off-chip sensors that take analog, physical measurements of the cluster (e.g. power or electromagnetic emissions) do not require trusting the operator. Such sensors could be retrofitted to existing datacentres (e.g. clamping a power meter on an input cable) and secured against tampering by the operator.

AI training has already been shown to leave a signature in the power drawn by the hardware that can serve as a diagnostic to identify and separate training from inference. A 10 MHz current probe clamped around the GPU's PCIe power supply cables can separate training from inference and from non-AI work at 97% accuracy (Gargiulo & Kulp, 2026), a result robust against many but not all adversarial strategies. On-chip 1 Hz telemetry counters that measure power (among other telemetry: utilisation, memory, etc.) can classify training at 95% accuracy, again a result robust against many but not all adversarial strategies (Rahman & Tajdari, 2026). In practice, any realistic AI governance agreements that use off-chip sensors will rarely have such fine-grained access to the GPU hardware. Power meters that can be retrofitted to existing facilities and secured against tampering are likely to sit at the level of the rack or facility input, with sampling frequencies $\sim$10 Hz (i.e. three to four orders of magnitude slower than the board probe) and behind a power-delivery path that low-passes and dilutes the signal. At this level training can be identified and it is large enough that datacentre and grid operators treat it as an engineering control problem to be dealt with (Patel et al., 2023; Ko & Zhu, 2025).

At the facility meter, the signature of training is its iteration cadence. Each iteration alternates a high-power compute phase with a low-power communication phase, about once a second, and this cycle puts a narrow line in the power spectrum. Detecting that line at a facility meter is harder than at the board, for two reasons. First, the cadence of a real run is not fixed. Stragglers, degraded hardware and temperature change the iteration time over seconds to hours (Section 2.1), so the spectral line wanders, and a test that looks for a sharp peak at one frequency loses it. Second, an operator who wants to hide can make the line wander on

[1]School of Mathematics and Statistics, The University of Melbourne, Parkville, VIC 3010, Australia [2]Department of Engineering Science, University of Oxford, Oxford, UK [3]Affiliation TBD. Correspondence to: Tom Kimpson <tom.kimpson@unimelb.edu.au>.

Preprint. October 6, 2026.

purpose, or smear it away, by inserting idle time between iterations or by varying the work done in each. Hiding is not free, however. Unlike a covert transmitter, a training operator cannot reshape its signal freely: every distortion of the schedule costs throughput, learning efficiency, or both. The question is therefore not whether training can be hidden, but whether it can be hidden cheaply.

In this paper we develop a method to detect training in facility-level power measurements against an operator who is trying to hide it, and we measure what hiding costs. We model the power of a datacentre running training and inference (Section 2) and pass it through a model of an off-chip meter (Section 3). To follow the wandering line we borrow a Viterbi line tracker from passive sonar and continuous-gravitational-wave searches (Section 4). Against an operator who does not hide, the tracker detects training in almost every five-minute window, within 0.04 of an optimal ceiling, while fixed-frequency tests fail once the cadence drifts (Section 5). It needs only a meter that samples at 2 Hz or faster and averages over no more than 0.5 s. We then test eight ways for an operator to hide (Section 6). Fixed-frequency tests are defeated cheaply: varying the work per iteration hides 88% of windows at no cost in throughput. The tracker is defeated only by extreme work variation, and that has a cost in learning instead: in small-scale training, the operator needs 93–267% more compute to reach the same loss. Over a day of observation, the strongest strategy we found escapes detection in 13% of runs (Section 7.1).

Our main conclusion is that a passive power meter outside the operator's control can detect synchronous training, and that hiding from it is expensive. Two limits apply. The meter detects a periodic schedule, not training itself, so non-training loads with a training-like cycle also trip it (Section 7.2). And our evidence is synthetic: the traces come from a model, not from a measured multi-GPU facility (Section 7.3).

This paper is organised as follows. Section 2 gives the model of datacentre training and inference power that generates every trace below, and Section 3 fixes what the verifier sees through an off-chip meter and what a concealing operator may do to hide from it. Section 4 develops the detector, a Viterbi line tracker; Section 5 reports its power against the inference null and the minimum meter it needs; and Section 6 prices what it costs a concealing operator to defeat it. Section 7 asks how likely a concealing operator is to be detected over a run, covers the non-training loads that imitate training, and sets out the limitations. Section 8 concludes.

2. A model of datacentre training power

In this section we build a model of the power draw from a data centre running AI workloads. This model will serve as a synthetic data generator to test our methods, in lieu of having access to real empirical data. We adopt the stochastic model of AI training power from Ko & Zhu (2025), developed for the analysis of grid stability, modified to add an Ornstein-Uhlenbeck drift on the training frequency. The reader is referred to Ko & Zhu (2025) for the full training model and its context.

This section is organised as follows. Section 2.1 describes the power draw from a training workload, following Ko & Zhu (2025), and adds our drift of its cadence. Section 2.2 describes the power draw from a fine-tuning workload. Section 2.3 describes the power draw from an inference workload. Section 2.4 combines these workloads into the load of a whole datacentre: one dominant workload, either training or inference, over a background of smaller trainings and fine-tunings. Table 1 collects every parameter value of the model.

2.1. Training

Multi GPU AI training of the kind that typifies frontier models is composed of a two-phase iterative cycle. First there is a power intensive, high compute "up" phase where the forward/backward pass happens. Second there is a low power, low compute, communication "down" phase where the gradient all reduce happens.

Iteration i has period

$$T_i \;=\; \frac{1}{f_0\,(1+\xi_i)}, \qquad \xi_i \sim \mathcal{N}(0,\sigma_\xi^2), \tag{1}$$

where f_0 is the baseline fluctuation frequency. For a given training run f_0 is fixed. Between training runs f_0 can vary depending on model size, batch size, parallelism layout, interconnect and hardware (Patel et al., 2024a; Jain et al., 2024). To describe the inter-workload variation, we take $f_0 \sim \mathcal{U}(f^{\mathrm{lo}}, f^{\mathrm{hi}})$. The period splits into an up-phase of duration $\rho_i T_i$ and a down-phase of duration $(1-\rho_i)T_i$, with the phase ratio $\rho_i \sim \mathcal{U}(\rho^{\mathrm{lo}}, \rho^{\mathrm{hi}})$ redrawn each iteration. Writing t for time within iteration i, $P(t)$ for the power the run draws above the facility's static overhead (added in Section 3.1), and P_{peak} for its peak,

$$\frac{P(t)}{P_{\mathrm{peak}}} = \begin{cases} 1+\Delta_i^{\mathrm{up}}+\nu^{\mathrm{up}}(t), & t<\rho_i T_i, \\ 1-\Delta_i^{\mathrm{dn}}+\nu^{\mathrm{dn}}(t), & t\ge\rho_i T_i, \end{cases} \tag{2}$$

where the per-phase level offsets are $\Delta_i^{\mathrm{up}} \sim \mathcal{N}(0,\sigma_\Delta^2)$ and $\Delta_i^{\mathrm{dn}} \sim \mathcal{N}(\mu_\Delta,\sigma_\Delta^2)$, and ν^{up}, ν^{dn} are zero-mean intra-phase Gaussian fluctuations with standard deviations $\kappa_\nu s_i^{\mathrm{up}}$ and $\kappa_\nu s_i^{\mathrm{dn}}$. The s_i^{up} and s_i^{dn} are redrawn each phase, uniformly from the ranges $[s^{\mathrm{up,lo}}, s^{\mathrm{up,hi}}]$ and $[s^{\mathrm{dn,lo}}, s^{\mathrm{dn,hi}}]$,

and κ_ν is a fixed scale factor. Table 1 lists the values of every parameter in this section.

Equations (1) and (2) are the exact training workload model from Ko & Zhu (2025). We make one modification, adding a slow drift to the cadence f_0 itself. The jitter ξ_i does not produce drift. Because it is redrawn independently each iteration, the cadence averaged over many iterations is still f_0. The timing errors accumulate as a random walk in phase, which broadens the spectral line into a Lorentzian of full width $\approx 2\pi\sigma_\xi^2 f_0$ (to leading order in σ_ξ) but leaves its centre at f_0. Real runs vary differently. Specifically, the iteration time changes in ways that persist. Every training step waits for the slowest GPU, so one slow component slows the whole run. In a production LLM cluster, these slowdowns last for the whole job rather than varying from step to step (Lin et al., 2025). Hardware can also keep running but degrade: CPU contention, overheating GPUs and congested network links cut throughput by tens of percent for periods of tens of seconds to hours (Wu et al., 2024). Over a day, temperature shifts throughput by a further 1–2% (Grattafiori et al., 2024).

We model this by letting the cadence of iteration i be $f_0 g_i$, where g_i is a discrete Ornstein–Uhlenbeck (first-order autoregressive) process that starts at $g_0 = 1$ and reverts to 1:

$$g_i - 1 \;=\; (1-\theta)\,(g_{i-1}-1) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma_g^2). \quad (3)$$

The period of Equation (1) becomes

$$T_i \;=\; \frac{1}{f_0\, g_i\, (1+\xi_i)}, \quad (4)$$

which reduces to Equation (1) when $g_i \equiv 1$. The stationary variance of Equation (3) is $\sigma_g^2/(2\theta-\theta^2)$, so we set

$$\sigma_g \;=\; \frac{\sigma_f}{f_0}\sqrt{2\theta-\theta^2}, \quad (5)$$

which makes the stationary standard deviation of the instantaneous cadence $f_0 g_i$ equal to σ_f, in Hz. The autocorrelation of g_i at lag k is $(1-\theta)^k$, so the cadence stays correlated over $\sim 1/\theta$ iterations. Over that span it wanders by σ_f, which can exceed both the line width set by ξ_i and the frequency resolution of a trace. We clip g_i from below at $g_{\min}$ so the period stays finite. We hold θ fixed and treat σ_f as a free parameter, sweeping it in Section 5.

2.2. Fine-tuning

Fine-tuning workloads can be represented using the same structure as in Section 2.1, with different parameters (Table 1). We denote the fine-tuning frequency as f_1. Compared with training, it has a lower cadence band, a longer tail phase (larger ρ_i), a deeper tail-to-idle swing (larger μ_Δ), and slightly smaller level offsets and intra-phase noise. We apply the cadence drift of Equation (3) only to the dominant run; fine-tuning and background training runs keep a fixed cadence ($\sigma_f = 0$).

2.3. Inference

Ko & Zhu (2025) do not present a model for power draw of inference workloads. We now construct one. We superpose three structures on a constant mean serving power load B. First, token generation (decode). Serving systems use continuous batching, which adds and removes requests from the batch at every step (Yu et al., 2022; Kwon et al., 2023). A per-step cadence exists, but the batch composition reshuffles every step. Unlike training, there is therefore no characteristic frequency: decode power smears across a broad band with no coherent frequency path. We model it as band-limited noise $b(t)$ of unit RMS over $[f_b^{\mathrm{lo}}, f_b^{\mathrm{hi}}]$, scaled by an amplitude A. Second, expert-load imbalance. Under sparse mixture-of-experts routing, tokens are not spread evenly across the experts (Shazeer et al., 2017), so the work per step varies with the mix of requests. We model this as a slow envelope on the decode power: a unit-variance Ornstein–Uhlenbeck process $u(t)$ with correlation time τ_u, entering at depth m. Amplitude modulation spreads the decode energy into sidebands, which makes the null harder, not easier. Third, prompt processing (prefill). Each request begins with a prompt phase that is brief, compute-bound, and draws more power than token generation (Patel et al., 2024a;b). We model these phases as box pulses $s(\cdot)$ of duration d_p and height a, arriving at times τ_k from a Poisson process of rate λ_p. These memoryless arrivals carry no line and simply raise the broadband in-band level. Together,

$$\frac{P^{\mathrm{inf}}(t)}{P_{\mathrm{peak}}} \;=\; B \;+\; A\,\big[1 + m\,u(t)\big]\,b(t) \;+\; \sum_k a\,s(t-\tau_k). \quad (6)$$

The forms of these structures follow how LLM serving systems schedule work; their amplitudes are our choice (Table 1). We set them so that inference is a loud confound rather than a quiet one. As Figure 1 shows, it delivers more in-band power than training but an order of magnitude less spectral contrast.

2.4. Aggregate

A datacentre rarely runs one job at a time. Following Ko & Zhu (2025) we write the load behind a single meter as the aggregate of one dominant training run over a background of smaller trainings and fine-tunings,

$$P_{\mathrm{AI}}(t) \;=\; P^{\mathrm{main}}(t) \;+\; \sum_{j=1}^{N_{\mathrm{tr}}} P_j^{\mathrm{tr}}(t) \;+\; \sum_{k=1}^{N_{\mathrm{ft}}} P_k^{\mathrm{ft}}(t). \quad (7)$$

where P^{main} is the power of the dominant training run, P_j^{tr} that of the j-th of N_{tr} smaller training runs, and P_k^{ft} that of the k-th of N_{ft} fine-tuning runs. Each term is an independent draw of Equation (2), with its own cadence, phase ratios, and level offsets. P^{main} and the P_j^{tr} use the training parameters and the P_k^{ft} use the fine-tuning parameters; only P^{main} carries the cadence drift of Equation (3). Peak power is split between the dominant run, the small trainings, and the fine-tunings in the ratio $w_{\text{main}} : w_{\text{tr}} : w_{\text{ft}}$, and each background share is divided equally among its members. Each small training draws its cadence from the same band as the dominant run, so the background carries weak lines of its own inside the search band.

We define the null aggregate as the same datacentre with the dominant training run replaced by inference (Section 2.3). The background is unchanged,

$$P_{\text{AI}}^{\text{null}}(t) \;=\; P^{\text{inf}}(t) \;+\; \sum_{j=1}^{N_{\text{tr}}} P_j^{\text{tr}}(t) \;+\; \sum_{k=1}^{N_{\text{ft}}} P_k^{\text{ft}}(t), \quad (8)$$

with P^{inf} from Equation (6) at the same share w_{main}. The two aggregate classes are then power-matched and both contain the background's weak lines. Figure 1 shows the four resulting workload classes as a verifier would see them.

	Symbol	Train	Fine-tune
Cadence band [Hz]	$[f^{\text{lo}}, f^{\text{hi}}]$	$[0.5, 1.5]$	$[0.3, 0.7]$
Period jitter	σ_ξ	0.1	0.1
Phase ratio	$[\rho^{\text{lo}}, \rho^{\text{hi}}]$	$[0.55, 0.8]$	$[0.7, 0.9]$
Offset spread	σ_Δ	0.05	0.03
Down-phase offset	μ_Δ	0.3	0.8
Up noise range	s^{up}	$[0.02, 0.05]$	$[0.01, 0.03]$
Down noise range	s^{dn}	$[0.01, 0.03]$	$[0.005, 0.02]$
Noise scale-down	κ_ν	0.15	0.15
Cadence drift (dominant run only)			
Reversion [iter^{-1}]	θ	0.02	
Excursion std [Hz]	σ_f	0 (swept)	
Drift floor	$g_{\min}$	0.1	
Aggregate			
Background counts	$N_{\text{tr}}, N_{\text{ft}}$	$4, 4$	
Power shares	$w_{\text{main}}{:}w_{\text{tr}}{:}w_{\text{ft}}$	$9 : 0.5 : 0.5$	
Inference null			
Mean serving load	B	0.5	
Decode RMS	A	0.40	
Decode band [Hz]	$[f_b^{\text{lo}}, f_b^{\text{hi}}]$	$[0.1, 4]$	
MoE corr. time [s]	τ_u	5	
MoE depth	m	0.4	
Prefill rate [Hz]	λ_p	0.08	
Prefill height	a	0.15	
Prefill duration [s]	d_p	0.1	
Scale			
Peak power [W]	P_{peak}	850	
Static overhead [W]	P_0	250	

Table 1. Workload-model parameters of Section 2. Training and fine-tuning values are Ko and Zhu's (Ko & Zhu, 2025), except κ_ν, the drift parameters, and σ_ξ for fine-tuning (Ko and Zhu list it for training only; we reuse it). The inference null is ours. Ko and Zhu's intra-phase noise is a sub-millisecond fluctuation, and each sample of our tens-of-hertz generator averages over many of them, so we scale its standard deviation by κ_ν; the s ranges are Ko and Zhu's values before scaling. Amplitudes are fractions of the peak power P_{peak}. The drift excursion σ_f is swept in Section 5. The scale values are ours, and are those of a single accelerator board; only the workload's size relative to the meter's additive terms affects detection (Section 3.1).

3. Measurement and threat model

This section specifies how the power drawn by the data centre reaches the verifier (i.e. the measurement model), and what the facility operator can control (i.e. the threat model). The verifier wants to know whether a monitored facility is running undeclared training. We frame this as a hypothesis test. Under the null hypothesis $\mathcal{H}_0$ the facility runs its declared workload, which for a monitored cluster is dominated by inference: the inference model of Section 2.3, alone or over the background of Equation (8). Under the alternative hypothesis $\mathcal{H}_1$ a training run of Section 2.1 takes the place of that inference, alone or over the same background (Equation (7)). Section 3.1 outlines the channel through which either hypothesis reaches the verifier. Section 3.2 outlines who controls the load, what they know, and what they may change.

3.1. The measurement model

Section 2 gives the power $P_{\text{AI}}(t)$ drawn by the AI workload above the facility's static overhead, in units of P_{peak}. The total power behind the meter is

$$P_{\text{fac}}(t) \;=\; P_0 + P_{\text{AI}}(t), \quad (9)$$

with P_{AI} given by Equation (7) under $\mathcal{H}_1$, by Equation (8) under $\mathcal{H}_0$, or by a single workload of Section 2 when a result drops the background. The verifier holds one scalar time series from a meter outside the operator's control. The meter reads

$$\begin{aligned} P_{\text{meter}}[n] \;&=\; (h * P_{\text{fac}})(t_n) + \eta[n], \\ t_n = n/f_s, \qquad h \;&=\; h_{\text{ins}} * h_{\text{del}}. \end{aligned} \quad (10)$$

The meter filters the facility power, samples it at rate f_s, and adds interference and measurement noise $\eta[n]$ on its own sample grid. Round brackets denote continuous time and square brackets index the meter's samples. The filter is a linear time-invariant system with impulse response h, which has two parts. The delivery path h_{del} is everything between the accelerators and the meter: power supplies, distribution, and any conversion or storage. The instrument h_{ins} is the meter's own filtering: its integration window and calibration gain. Figure 2 traces the power from the workloads to the

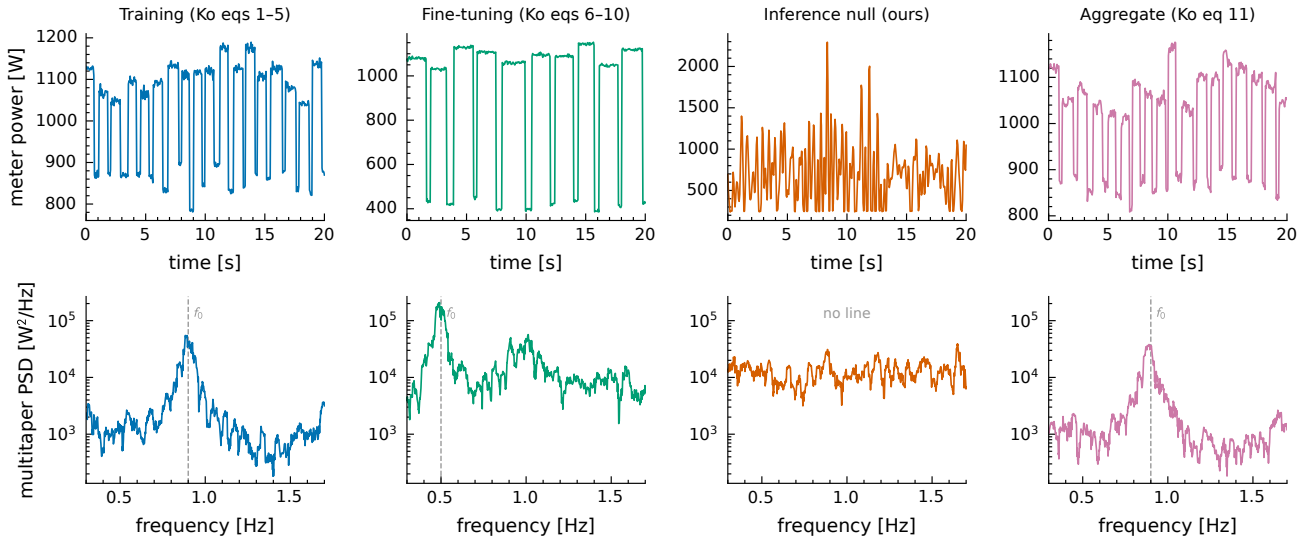


Figure 1. The four workload classes at the power meter, on the nominal channel (20 Hz sampling, no filtering or decimation, $\sigma_\eta = 4$ W). Top: a 20 s excerpt. Bottom: the multitaper spectrum over the 0.3–1.7 Hz search band, with the true cadence marked. Training, fine-tuning, and the aggregate carry a line at f_0; the inference null does not. Cadences are pinned for legibility ($f_0 = 0.9$ Hz training, 0.5 Hz fine-tuning); elsewhere they are drawn from the Ko bands.

verifier.

Following Ko & Zhu (2025) we assume the delivery path passes the power through unchanged, such that h_{del} is a delta function and $h_{\text{del}} * P_{\text{fac}} = P_{\text{fac}}$. The low-pass and notch of Table 2 relax this assumption in our sweeps. The notch stands for an anti-resonance of the delivery network that happens to sit near the cadence; it is the one stage that removes signal in-band while leaving the rest of the spectrum intact.

The instrument h_{ins} is the meter's own response (Table 2). We model it as a calibration gain g followed by a trailing boxcar, so that each reading is the mean power over the preceding window τ: $h_{\text{ins}}(t) = g/\tau$ for $0 \le t < \tau$ and zero otherwise. Its frequency response has magnitude

$$|H_{\text{ins}}(f)| \;=\; g\left|\frac{\sin(\pi f \tau)}{\pi f \tau}\right|, \tag{11}$$

which passes slow variations and attenuates fast ones, with a zero at every multiple of $1/\tau$. A window longer than about 0.6 s puts the first zero inside the 0.3–1.7 Hz search band, and a 1 s window puts it at the band centre. In the nominal channel $g = 1$ and $\tau \to 0$, so h_{ins} is also a delta function, and the meter samples at $f_s = 20$ Hz. Unlike the delivery path, the instrument is the verifier's to choose, and Section 5.2 asks what τ and f_s must be for detection to hold.

The additive term $\eta[n]$ collects everything that reaches the readings without passing through h (Table 2). In the nominal channel it is white meter noise of $\sigma_\eta = 4$ W. Two further terms are available: a slowly wandering baseline, standing for a drifting operating point such as cooling load, and a controller limit cycle, a square wave whose frequency itself wanders, modelled on the power-management oscillation measured on an A100 (Section 7.2). The controller is the first appearance of an instrument line: a persistent, slowly wandering spectral line that the facility's hardware produces with no training present. Section 7.2 returns to it.

Three numbers carry units. The peak power $P_{\text{peak}} = 850$ W and overhead $P_0 = 250$ W (Table 1) are those of a single accelerator board, and the meter noise $\sigma_\eta = 4$ W is an illustrative choice, about 0.5% of the mean reading, not a measured value. None of the three is critical. P_0 is constant, so it lies at zero frequency, outside the search band. What sets detectability is the training line against everything else in the band, and at the nominal setting that is dominated by the workloads themselves: in the search band the inference null's power spectral density exceeds that of the meter noise by about four orders of magnitude, and the training trace's own off-line floor exceeds it by about three. Meter noise binds only when it is raised far above 4 W, which Figure 3 does over two decades.

Each trace lasts $T = 300$ s, a frequency resolution of $1/T \approx 3$ mHz. In the nominal channel every stage of Table 2 is off except the white meter noise. With the noise also off, the map returns P_{fac} unchanged and consumes no randomness. Two limits apply. The linear time-invariant form of h is a first approximation; a real delivery path may be operating-point dependent. And the model quantises only in time, through the integration window and the sample grid; it has no amplitude quantisation. Until simultaneous on-device and external-meter measurement is performed,

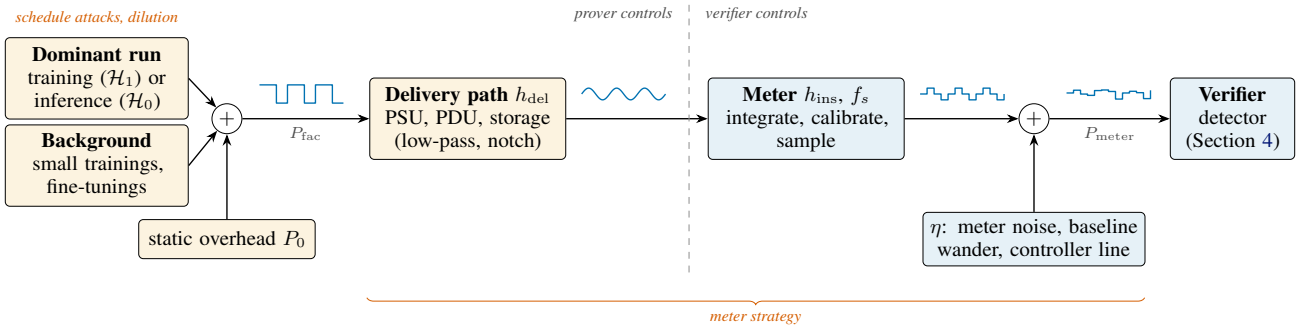


Figure 2. The measurement chain. The workloads of Section 2 and a static overhead sum to the facility power P_{fac} (Equation (9)), which passes through the delivery path and the meter before additive terms are added (Equation (10)). Sketches show one training cadence as it is smoothed, sampled, and made noisy; they are illustrative. Everything left of the dashed line is under the operator's control (Section 3.2). Red labels mark where the adversarial strategies of Table 3 act. η lumps instrument noise with interference that physically originates behind the meter, such as the power-controller line. In the nominal channel the delivery path passes the power through unchanged ($h_{\text{del}} = \delta$) and the meter samples at 20 Hz with no integration (Table 2).

claims about what reaches an off-chip sensor are predictions (Section 7.3).

Figure 3 shows how the channel attenuates the training line. For a fixed training workload we vary one axis of Table 2 at a time and measure an effective signal-to-noise ratio at the cadence: the spectral power at f_0 over the in-band power the meter itself adds, relative to the nominal channel. Every axis lowers it, and two remove the line entirely within the range a real meter might occupy: a 1 s integration window, whose first zero falls at 1 Hz (Equation (11)), and a 1 Hz sample rate. This concerns the signal reaching the verifier, not any detector; what each detector does with what survives, and hence the minimum meter specification, is Section 5.2.

3.2. The threat model

The verifier installs and owns the power meter. The meter is tamper-evident, the verifier chooses its specification, and the operator cannot forge the meter noise η. The operator controls everything behind the meter, e.g. which jobs run, how they are scheduled, what else shares the facility, and the power-delivery path itself. Energy storage or power smoothing between the accelerators and the meter can low-pass or notch the load, and datacentre and grid operators already treat smoothing training load as an engineering problem (Ko & Zhu, 2025). We represent partial smoothing by the filtering stages of Table 2, and Section 5.2 asks how much of it the verifier can tolerate. A store large enough to flatten the load entirely would erase any passive signature; we do not model it, and discuss it among the limitations (Section 7.3). The operator knows the detector, its parameters, its threshold, and the meter specification. It does not know the realisation of η.

We consider three types of facility operator. First a *compliant* operator runs only its declared inference. A *benign* operator trains without regard to being caught by the power meter. A *concealing* operator trains, knows the detector, and reshapes its workload to try to evade it. The concealing operator's objective is to train without the being detected by the verifier. They use different adversarial strategies to try to subvert the verifier's detection. Table 3 lists the adversarial strategies considered in this work. They fall into five categories: disturbing the timing of iterations, relocating the cadence, reshaping the power waveform within an iteration, mixing the run with other load, and degrading the channel. In principle the strategies may be composed, but we treat them individually. Adversarial strategies have a cost to the operator in terms of training throughput. Each strategy therefore trades concealment against cost.

Three things lie outside this threat model. First, the operator does not tamper with the meter or forge its noise. Second tt does not run a semantic decoy, that is a load that reproduces the physical schedule of training without training, since manufacturing evidence of the activity it is hiding buys a concealing operator nothing. Finally, it does not move the training cadence outside the searched band or abandon synchronous iteration altogether; both are scope restrictions on the population we target (Section 7.3).

4. Detection method

In this section we present a detection method that ingests $P_{\text{meter}}[n]$, Equation (10), and decides between $\mathcal{H}_0$ and $\mathcal{H}_1$ (Section 3). The two hypotheses are separated by the iteration cadence of synchronous training. Under $\mathcal{H}_1$ the two-phase cycle of the dominant run (Equation (2)) puts a strong, narrow spectral line at its cadence f_0. Under $\mathcal{H}_0$ the inference that replaces it spreads its power across the band with no line (Section 2.3), and the only lines left are the background's weak ones (Section 2.4). A line at a fixed frequency has classical tests. But the cadence of a real run wanders by σ_f even without an adversary (Equation (3)),

Stage	Parameter	Nominal	Swept	Stands for
Delivery path (h_{del}; the identity in the workload model)				
Butterworth low-pass (order 2)	cutoff [Hz]	off	8–0.5	PSU/PDU bandwidth, smoothing
In-band notch	depth; centre [Hz]; Q	off	0.5–1; 0.6–1.2; 1.5	delivery-path anti-resonance
Instrument (h_{ins} and sampling; chosen by the verifier)				
Trailing boxcar integration	window τ [s]	0	0.1–2	meter averaging
Gain	g	1	—	calibration factor
Sampling (no anti-alias filter)	f_s [Hz]	20	10–0.5	meter reporting rate
Additive terms (η)				
Meter noise (white or AR(1))	σ_η [W]; τ_η [s]	4; white	1–120; 2	instrument noise
Baseline wander (OU)	std [W]; corr. time [s]	off	20; 60	drifting operating point
Controller limit cycle	amplitude [W pk-pk]	off	10–120 at 0.38 Hz	power-management loop

Table 2. Stages of the observation map (Equation (10)). Nominal values define the channel used everywhere unless stated. The swept ranges are those of Figure 3 and the minimum-meter sweep of Section 5.2; the hostile channels of the meter strategy (Table 3) are drawn from the same ranges. Every stage off except η is the nominal channel; every stage off including η is the identity.

and a concealing operator can make it wander further or vary the work per iteration, which defeats a fixed-frequency test cheaply (see Section 6). The detector therefore has to follow the line as it moves.

This section is organised as follows. Section 4.1 defines the detector, a Viterbi line tracker borrowed from passive sonar and continuous-gravitational-wave searches. Section 4.2 sets the threshold at which the detector declares training. Section 4.3 defines the two fixed-frequency tests we compare it against.

4.1. Viterbi line tracking

We follow the line with the hidden-Markov frequency tracking developed for passive sonar (Streit & Barrett, 1990) and carried into continuous-gravitational-wave searches (Suvorova et al., 2016; Bayley et al., 2019; Djurović & Stanković, 2011). The tracker has three parts: a spectrogram of the trace, a contrast map built from it, and a search for the best slowly varying path through that map.

Let $x[n] = P_{\mathrm{meter}}[n] - \bar{P}_{\mathrm{meter}}$ be the mean-removed meter trace, $N = f_s T$ samples long. We cut it into frames of M samples that overlap by half, a hop of $H = M/2$, and take the power spectrum of each:

$$S_{kj} \;=\; \left| \sum_{m=0}^{M-1} w[m]\, x[jH+m]\, e^{-2\pi i k m/M} \right|^2, \qquad (12)$$

for frames $j = 0, \ldots, N_t - 1$, where w is a tapered (Tukey) window and $N_t = \lfloor (N-M)/H \rfloor + 1$ is the number of frames. Bin k sits at frequency $f_k = k f_s/M$. We choose M so that each frame lasts $M/f_s = 16\,\mathrm{s}$, which gives bins $\Delta f = 1/16 = 62.5\,\mathrm{mHz}$ wide, fine enough to resolve a line near 1 Hz, while leaving many frames to track over: a 300 s trace gives $N_t = 36$ frames, 8 s apart. Only the bins with $f_k \in \mathcal{B}$ enter the tracker, 23 of them in the nominal channel. Every detector in this paper searches the band $\mathcal{B} = [0.3, 1.7]$ Hz. This is the training cadence band $[f^{\mathrm{lo}}, f^{\mathrm{hi}}] = [0.5, 1.5]$ Hz of Table 1, widened by 0.2 Hz on each side so that a cadence drawn near either edge stays in the band as it wanders.

We normalise each frame by its median over the band,

$$L_{kj} \;=\; \ln S_{kj} \;-\; \ln \tilde{S}_j, \qquad (13)$$

where $\tilde{S}_j$ is the median of S_{kj} over the in-band bins of frame j. L_{kj} measures how far bin k stands above the typical in-band level of its own frame. The normalisation keeps a slow change in overall power from masquerading as a line.

The hidden state is the frequency bin occupied by the cadence in each frame. The prior that a real cadence wanders slowly enters as a penalty λ per bin of frequency change between consecutive frames. The detection statistic is the score of the best path, averaged over frames,

$$V \;=\; \frac{1}{N_t} \max_{k_0,\ldots,k_{N_t-1}} \left[\sum_{j=0}^{N_t-1} L_{k_j j} \;-\; \lambda \sum_{j=1}^{N_t-1} |k_j - k_{j-1}| \right], \qquad (14)$$

with $\lambda = 1$ throughout. A path pays one unit of log-contrast for each 62.5 mHz step it takes between frames, so it can follow a drift of about one bin per frame (about $8\,\mathrm{mHz\,s^{-1}}$) if the line it follows is bright enough to pay for it. Dynamic programming evaluates the maximum over all 23^{N_t} paths exactly, in time linear in N_t. A drifting line keeps a high score because the path bends to follow it. Broadband power, such as the continuous-batching floor of the inference null, does not, because no single slowly varying path stays on top of it. Figure 4 shows both cases on the contrast map. Averaging over frames makes V comparable across traces of different length. Note that V tests whether a persistent, slowly wandering line is present, not what produced it. Accordingly, strong lines that do not have a training provenance may also trigger the detector; this is addressed in Section 7.2

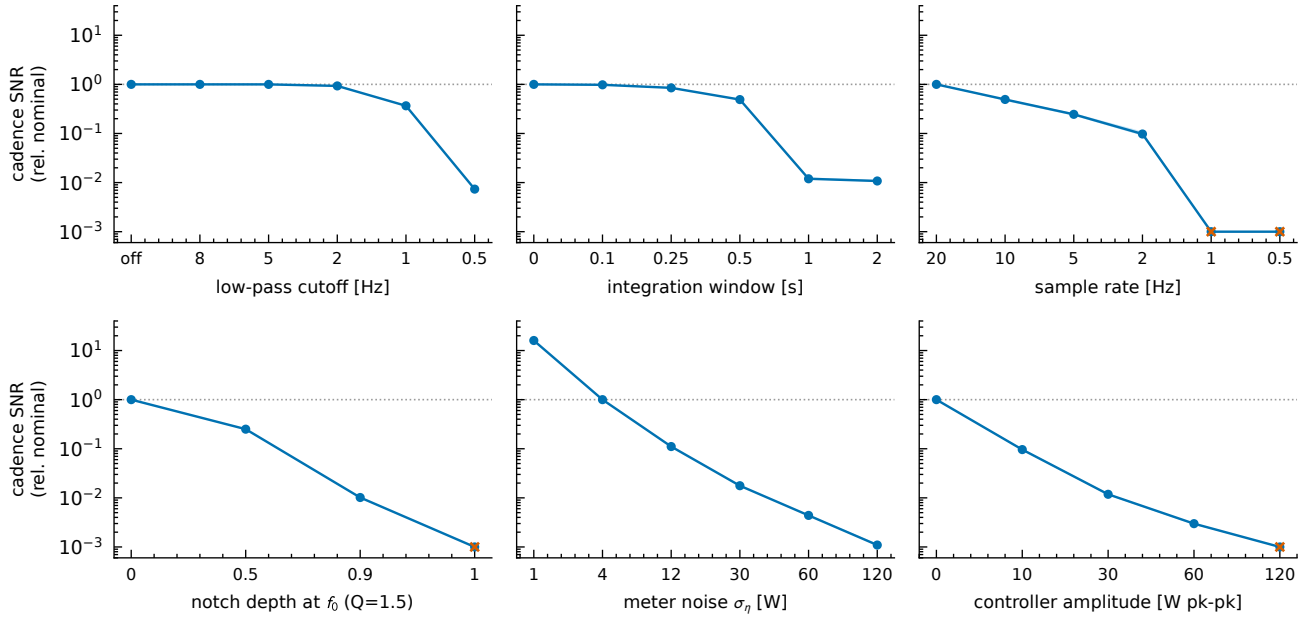


Figure 3. Sensitivity of the cadence signature to the observation map, one axis at a time on a fixed honest training workload ($f_0 = 0.9$ Hz, 20 realisations, median and interquartile range). The measure is spectral power at the known f_0 over the in-band power the meter itself contributes, normalised to the nominal channel, so that attenuating axes act on the numerator and additive ones on the denominator and every panel reads the same way: down is worse for the verifier. Crosses mark settings where the cadence is not observable at all. Note f_0 is pinned away from 1.0 Hz, which would coincide with both the 2 Hz Nyquist frequency and the first sinc null of a 1 s boxcar.

4.2. Setting the threshold

The tracker declares training when V exceeds a threshold v_α, chosen so that the false-alarm rate is α:

$$\Pr\big(V > v_\alpha \mid \mathcal{H}_0\big) \;=\; \alpha. \tag{15}$$

We use $\alpha = 10^{-2}$ unless stated. V has no analytic null distribution. We therefore set v_α empirically, as the $(1-\alpha)$ quantile of V over simulated traces drawn under $\mathcal{H}_0$, and measure detection rate as the fraction of $\mathcal{H}_1$ traces above it. The baseline methods of Section 4.3 are thresholded the same way.

In Section 5 the $\mathcal{H}_0$ population is the null aggregate of Equation (8). Its background carries the weak lines of the small trainings and fine-tunings, so the threshold already accounts for any line that is not the dominant run's. Calibrating on inference alone, with no background, changes the tracker's detection rates by at most 0.02 (Appendix A).

This calibrates V against the modelled inference null. A deployed verifier has to obtain the same threshold from data it holds: from background traces known to be free of training, as continuous-wave searches set their tracking thresholds from noise-only realisations (Suvorova et al., 2016), or from surrogates generated from each observed trace.

4.3. Baseline method

Two simple fixed-frequency tests serve as baselines to compare our Viterbi method against. The first is a spectral matched filter, which uses the peak of the power spectral density in the band, relative to its median over all frequencies,

$$D_{\mathrm{MF}} \;=\; \frac{\max_{f\in\mathcal{B}} \hat{S}(f)}{\mathrm{median}_f\, \hat{S}(f)}, \tag{16}$$

where $\hat{S}$ is a Welch estimate of the spectrum of $x[n]$ from half-overlapping Hann-windowed segments of 1024 samples (51.2 s at $f_s = 20$ Hz). The long segments concentrate a stationary line into one bin, which is the test's strength; a wandering line smears across many bins and the peak collapses. The second is the Thomson multitaper harmonic F-test (Thomson, 1982; Percival & Walden, 1993), maximised over the band,

$$D_{\mathrm{F}} \;=\; \max_{f\in\mathcal{B}} F(f), \tag{17}$$

where, at any fixed f under a stationary, locally smooth null, $F(f) \sim F(2, 2K-2)$, with $K = 7$ tapers of time–bandwidth product $NW = 4$. $F(f)$ is the ratio of the power in a line at f to the power left around it (Equation (21), Appendix B).

Strategy	What the operator changes	Budget knob (range)	Cost anchor
Timing			
Period jitter	i.i.d. period jitter σ_ξ of Equation (1), by inserting idle time	$\sigma_\xi \in [0.05, 0.7]$	measured
Cadence drift	OU excursion σ_f of Equation (3)	$\sigma_f \in [0, 1.5]$ Hz	measured
Work variation	micro-steps per iteration G_i, drawn about $G_0 = 8$ with fractional spread σ_G; down-phase unchanged	$\sigma_G \in [0.05, 0.7]$	measured
Phase slip	Brownian slip of iteration boundaries at fixed mean period	slip std $\in [0.01, 0.2]$ periods	—
Frequency			
Relocation	cadence f_0 moved toward the search-band edges	$f_0 \in [0.35, 1.65]$ Hz	—
Waveform			
Harmonic smoothing	raised-cosine ramp on phase transitions; comb dies, fundamental survives	ramp width $\in [0.1, 1]$ s	—
Shape filling	down-phase level raised toward the up-phase at matched mean power	fill fraction $\in [0, 1]$	measured
Mixing			
Dilution	training's share of the aggregate (Equation (7)) lowered	share $\in [0.2, 0.9]$	—
Channel			
Meter	hostile h and η: notch at the cadence; 1 s/1 Hz integrating sampler; in-band controller; heavy coloured noise	four named channels	—

Table 3. Adversarial strategies available to the concealing operator. Each is implemented inside the generator of Section 2 and swept singly over the stated budget range in Section 6. "Measured" marks strategies with a throughput-overhead anchor from the single-A100 campaign; the others were not run on hardware and are reported without a cost (Section 6). The meter strategy degrades the channel rather than the schedule and is read as a channel requirement (Section 5.2), not priced.

5. Detection results

In this section we test the Viterbi tracker of Section 4 on synthetic meter traces. We assess if the Viterbi tracker can detect the training frequency, and what sampling settings are required by the power meter for the Viterbi tracker to work. We treat the operator as benign; an operator who reshapes the schedule to hide the line is the subject of Section 6. Under $\mathcal{H}_1$ the meter sees a dominant training run over $N_{\text{tr}} = 4$ small trainings and $N_{\text{ft}} = 4$ fine-tunings (Equation (7)). Under $\mathcal{H}_0$ it sees the null aggregate, with inference in the dominant slot over the same kind of background (Equation (8)). The shares are the defaults of Table 1, $9 : 0.5 : 0.5$. Both classes therefore also carry the background's weak lines, and the tracker has to find the dominant one among them.

This section is organised as follows. Section 5.1 measures the tracker's detection rate as the cadence drifts, and compares it with the fixed-frequency tests of Section 4.3 and with an optimality ceiling. Section 5.2 sweeps the meter and states the minimum specification under which detection holds.

5.1. Detection rate under cadence wander

We consider a range of cadence-drift $\sigma_f \in \{0, 0.1, 0.2, 0.4, 0.8, 1.5\}$ Hz (Equation (5)) in the dominant training run, with $n = 200$ traces of 300 s per class and thresholds set at an empirical false-alarm rate of 10^{-2}. We benchmark the Viterbi method against a Neyman–Pearson optimal ceiling (Appendix A). Figure 5 shows the result. The Viterbi tracker stays within 0.04 of the optimal ceiling across the whole σ_f axis, with detection rate 1.00 up to 0.4 Hz of drift, 0.99 at 0.8 Hz and 0.96 at 1.5 Hz. Conversely, the baselines fixed tests struggle. The spectral matched filter detects a stationary line (0.98 at zero drift) but falls to 0.59 at 0.1 Hz and 0.23 at 0.4 Hz, and the multitaper F-test has essentially no power at empirical thresholds at any drift (≤ 0.03).

5.2. A minimum meter specification

The results of Section 5.1 assume the nominal meter of Table 2: a sample rate of 20 Hz, no integration window ($\tau \to 0$), no filtering on the delivery path, and white meter noise of $\sigma_\eta = 4$ W. A real meter may sample more slowly, average over a longer window, or sit behind a delivery path that suppresses the cadence band. The meter is the verifier's choice, so it is of interest to find the minimum specification it must meet for detection to hold. We sweep the sample rate over $f_s \in \{20, 10, 5, 2, 1, 0.5\}$ Hz and the integration window over $\tau \in \{0, 0.1, 0.25, 0.5, 1, 2\}$ s (Equation (11)). We also sweep a notch in the delivery path, i.e. a band near the cadence in which the delivery path suppresses the power swing (Section 3.1). Its centre ranges over 0.6–1.2 Hz and its depth, the fraction of the signal it removes at the centre, over 50–100%, with the meter otherwise nominal. Throughout, the honest aggregate is scored against the null aggregate, with thresholds at an empirical false-alarm rate of 10^{-2} as in Section 5.1.

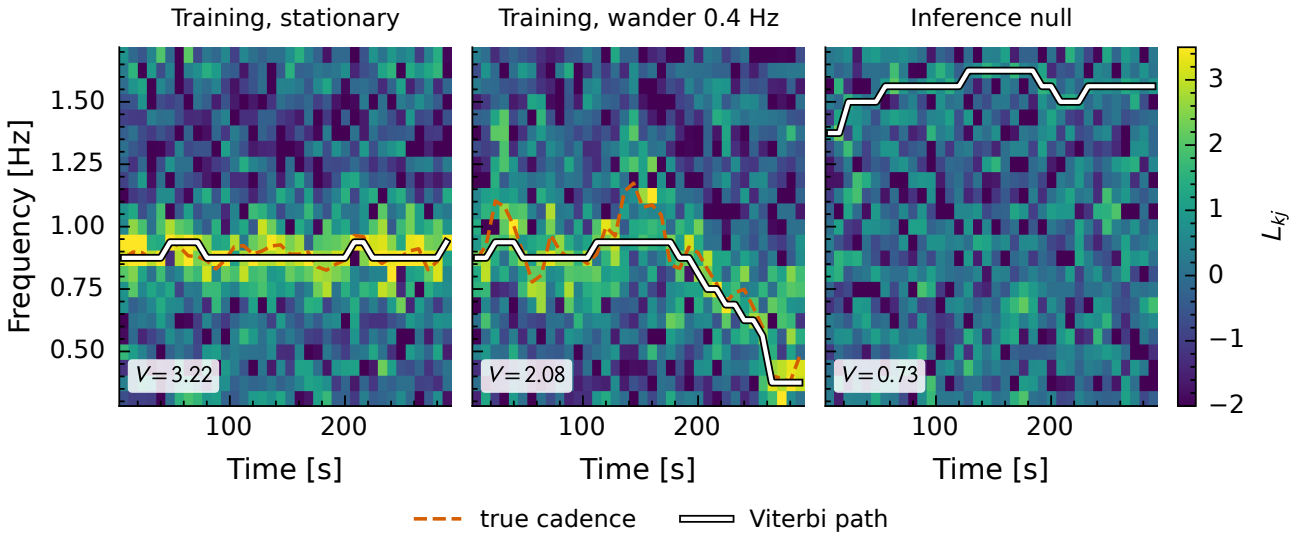


Figure 4. The Viterbi tracker on three 300 s traces at the nominal channel. Each panel is the contrast map L_{kj} of Equation (13) on the tracker's own grid (16 s frames at half overlap, 62.5 mHz bins over $\mathcal{B}$), with the best path of Equation (14) in white and its score V inset. Dashed: the true cadence, $1/$(median iteration period) in each frame, from the generator's iteration boundaries. Left: a stationary training cadence ($f_0 = 0.9$ Hz), which the path sits on. Centre: the same run with 0.4 Hz of cadence wander, which the path follows across most of the band; it cuts the corner off the brief excursion near 150 s, where following the line costs more in jumps than it gains in contrast. Right: the inference null, where no path finds sustained contrast and V falls to 0.73.

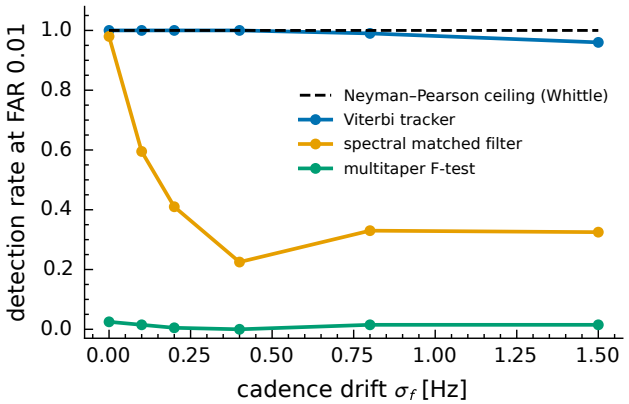


Figure 5. Detection rate at empirical false-alarm rate 10^{-2} against the null aggregate, versus cadence drift σ_f of the dominant training run ($n = 200$ per class, 300 s traces, shares $9 : 0.5 : 0.5$). The Viterbi tracker stays within 0.04 of the Whittle Neyman–Pearson ceiling across the wander axis; the spectral matched filter collapses once the line moves and the multitaper F-test has no usable power. The ceiling is described in Appendix A.

The Viterbi tracker holds full power (detection rate $\gtrsim 0.9$) if and only if the meter satisfies three conditions together (Figure 6), one on each part of Equation (10). First, on the sampling, a rate of at least about 2 Hz: rates of 20, 10, 5, and 2 Hz all hold the Viterbi tracker near 1.0, 1 Hz collapses it to 0.33 as the Nyquist edge crosses the band, and 0.5 Hz is dead (every detector at chance). Second, on the instrument's impulse response, an integration window $\tau \lesssim 0.5$ s: at 20 Hz sampling $\tau = 0.5$ s still holds (0.94), but $\tau = 1$ s puts the first zero of Equation (11) at $1/\tau = 1$ Hz, the band centre, and collapses the tracker to 0.32 even at full sample rate. Third, on the delivery path, no deep transfer-function notch at the cadence: a shallow notch, or one near the edge of the cadence band, costs little (tracker ≥ 0.80), but a notch of 90–100% depth at 1.0–1.2 Hz roughly halves tracker power (to 0.45–0.63) without erasing it (Appendix C).

Against an honest operator, then, detection needs only a meter that samples at $f_s \geq 2$ Hz, integrates over $\tau \leq 0.5$ s, and has no deep notch at the cadence.

6. Detection against a concealing operator

Section 5 assumed a benign operator, who leaves the training cadence for the tracker to find. A concealing operator can instead de-periodicise its schedule and smear the cadence away (Section 3.2). In principle this is always possible. The concealing operator can insert idle time at random between iterations, or vary the work done in each, to remove any frequency for the Viterbi detector to identify. The question is the cost in training throughput the operator must incur in order to de-periodicise enough to hide from the Viterbi detector, and in particular whether any de-periodicisation strategy is both cheap and effective.

In this section we sweep all eight schedule strategies of Table 3 and measure how much each one hides from the detectors. The operator applies each strategy to its dominant

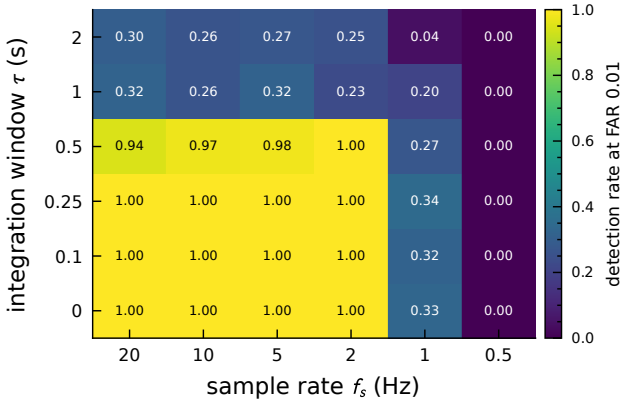


Figure 6. Minimum meter specification: Viterbi detection rate at empirical false-alarm rate 10^{-2} over sample rate $f_s \times$ integration window τ, the honest aggregate against the null aggregate ($n = 200$ per class). The live region is $f_s \geq 2$ Hz *and* $\tau \leq 0.5$ s.

training run, P^{main} in the aggregate of Equation (7). The background of small trainings and fine-tunings, P_j^{tr} and P_k^{ft}, stays honest, and every cell is scored against the aggregate null of Equation (8). For each strategy we step its budget knob over the range in Table 3. We measure the detection rate of the Viterbi tracker and of the fixed tests of Section 4.3, taking the better of the two fixed tests at each cell. All rates are at false-alarm rate 10^{-2} with $n = 200$ traces per class.

We measure the throughput cost incurred by four of the concealing strategies (period jitter, cadence drift, work variation and shape filling) on a single A100. We write the throughput cost as

$$c = \frac{r_{\text{honest}}}{r} - 1, \tag{18}$$

i.e. the fractional increase in wall-clock time per trained token, where r is the training throughput in tokens per second under the concealing strategy and r_{honest} that of the same run without it. For the strategies that only insert idle time, every iteration trains the same number of tokens, so c is the idle time added per iteration as a fraction of the busy time. Work variation changes the number of tokens per iteration, so there we measure r directly in tokens per second. Appendix D gives the measurement details. We did not run the other four concealing strategies (phase slip, relocation, harmonic smoothing and dilution) on hardware, so they have no measured cost.

Figure 7 plots hiding (1− detection rate) against the budget knob (and, where applicable, the measured throughput cost) of every concealing strategy Against the fixed baseline detection methods, seven of the eight strategies hide a large fraction of runs at their larger budgets; only harmonic smoothing hides nothing. Hiding mostly grows with the budget, up to 0.88 under both jitter and work variation. Only work variation hides for free. It hides 0.88 at $c \approx 0$, while jitter and drift need $c = 159\%$ and 356% to reach 0.88 and 0.71. Against the Viterbi tracker almost nothing hides. The tracker detects at least 95% of runs in 43 of the 46 cells. It detects every run in every cell of phase slip, relocation, harmonic smoothing, shape filling and dilution, and at least 97% under drift. The three exceptions are two jitter settings and the largest work variation. Jitter at $\sigma_\xi = 0.35$ hides 0.24 for $c = 159\%$, and at 0.5 hides 0.12 for $c = 375\%$. Work variation at $\sigma_G = 0.7$ hides 0.55 at $c \approx 0$, though at every smaller σ_G it hides at most 0.05.

Work on real hardware prices de-periodicisation the same way as our measured costs, through a different channel. Four evasion strategies run on a single H200 — chunked optimizer updates, dilution with decoy inference, throttling, and diluted LoRA — cost between 28 and 69% of throughput (Gargiulo & Kulp, 2026). Each inserts or substitutes work, which is what makes it expensive, and none varies the real computation per iteration, the one strategy we find cheap.

While the Viterbi detector is robust to the majority of concealing strategies, work variation at $\sigma_G = 0.7$ is the main exception; the operator can hide 55% of 300 s observation windows at zero throughput cost. However, the strategy does have a cost in terms of training efficiency. The work variation strategy involves a few iterations accumulating many micro steps, where about one in ten accumulates 64 microsteps or more. The long blocks raise the mean microstep count from 8 to 25, so the operator takes about a third as many optimiser steps per token as honest training, and each step on a larger batch does less per token. McCandlish et al. (2018) model an optimiser step on a batch of B samples, with the learning rate tuned for that batch, as making a fraction

$$h(B) = \frac{B}{B + B_{\text{noise}}} \tag{19}$$

of the progress of a full-gradient step, where the gradient noise scale B_{noise} sets the critical batch size. For microstep counts G_i drawn independently, progress per sample is proportional to $\mathbb{E}[h(G)]/\mathbb{E}[G]$, so the attack needs a fraction

$$\Delta = \frac{h(G_0)/G_0}{\mathbb{E}[h(G)]/\mathbb{E}[G]} - 1 \tag{20}$$

more samples than honest training to reach the same loss, with B and B_{noise} counted in micro-batches; at $\sigma_G = 0.7$ it gives $\Delta = 53$–184% for B_{noise} between 8 and 128.

We also measured the training efficiency penalty for the work variation strategy directly. We trained a 30M-parameter GPT-style language model on 400M tokens of FineWeb-Edu (Penedo et al., 2024) on a single A100, three seeds per schedule. The strategy uses the same micro-batches in the same order as honest training ($G_0 = 8$), changing only how many are accumulated into each step; the gradient noise scale we measured over these runs is

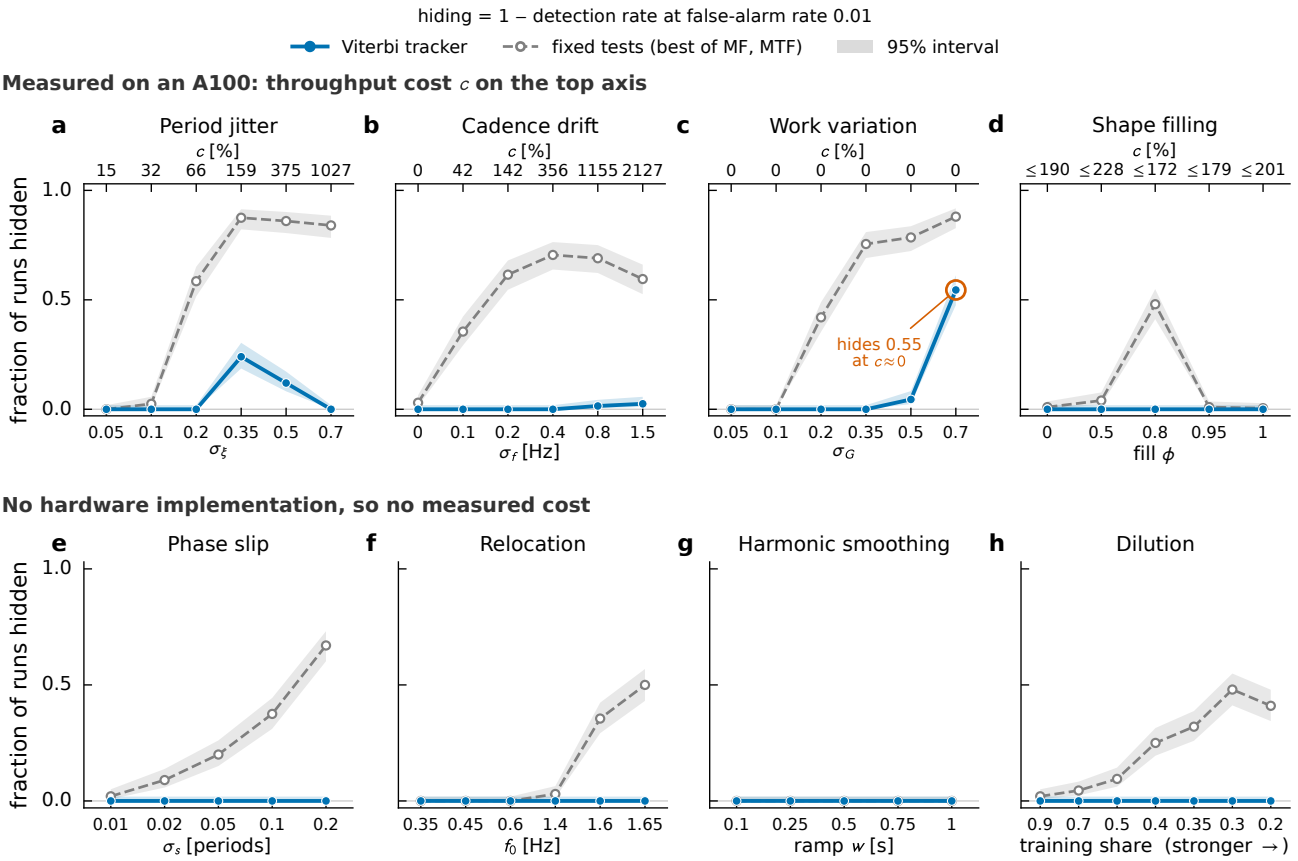


Figure 7. How much each schedule strategy hides, and what it costs. Each panel steps one strategy of Table 3 through its budget knob (bottom axis; settings evenly spaced, weakest on the left) and plots hiding, $1-$ detection rate at false-alarm rate 10^{-2}, for the Viterbi tracker (blue, solid) and the fixed tests (grey, dashed; the better of the matched filter and multitaper F-test per cell). Bands are Wilson 95% intervals for $n = 200$ traces per class. **a–d**, the four strategies run on a single A100. The top axis gives the measured throughput cost c of each setting (Equation (18)); costs below 1% in magnitude, zero within noise, are shown as 0, and drift-free scheduling costs zero by definition. Shape-filling costs were measured on a jittered base and are upper bounds ($\leq$). **e–h**, the four strategies we did not run on hardware, and so with no measured cost. The tracker detects almost every cell. The exception, ringed in **c**, is work variation at $\sigma_G = 0.7$, which hides 0.55 from it at $c \approx 0$; idle-insertion jitter hides at most 0.24, for $c = 159\%$. The meter strategy degrades the channel rather than the schedule and is treated in Section 5.2.

about 14 micro-batches. We define the learning cost as the extra tokens the strategy needs, i.e. the 400M tokens the strategy trained on, divided by the tokens honest training needs to reach the same final loss, minus one. At $\sigma_G = 0.7$ it is $267 \pm 33\%$, c.f. the predicted 163% at the measured noise scale from Equation (20). We also tried a strategy not in Table 3: draw G_i lognormally with spread s, with the mean held at the honest 8. It takes as many optimiser steps per token as honest training and never idles, so it should cost no throughput. At $s = 1.0$ it hides 0.73 from the tracker, more than the 0.55 of work variation at $\sigma_G = 0.7$, but costs $93 \pm 6\%$ in training efficiency.

7. Discussion

Sections 5 and 6 show that a Viterbi tracker detects synchronous training against a declared inference workload, given a meter that samples at ≥ 2 Hz with ≤ 0.5 s integration. The method is robust against the majority of concealing strategies, while the minority of strategies that hamper detection pay a cost in either throughput or learning efficiency. In this section we discuss the implications of these results for a verifier who is monitoring a facility for signs of training. This section is organised as follows. Section 7.1 extends the short, per-window detection rates of Sections 5 and 6 into the probability that a concealing operator is detected over a long training run. Section 7.2 considers confounding, non-training workloads that imitate training and that the tracker also detects. Section 7.3 discusses limitations of the method and our analysis, and suggests avenues for future work.

7.1. Detection probability for long runs

Every detection rate in Section 6 refers to a single 300 s window. However, a frontier training run lasts days to months. The verifier observes the power meter over these longer timescales. The quantity that both the verifier and the concealing operator are interested in is the is the probability of being detected at some point during these longer runs. In this section we compute this detection probability.

For the majority of concealing strategies the Viterbi detector detects training in at at least three quarters of windows, and in most settings in nearly all of them, so a run is easily caught. We therefore consider exclusively the work variation strategy, which was the only strategy that could hide a large fraction of windows from the tracker. We take two forms of work variation, the $\sigma_G = 0.7$ setting of Table 3 and the lognormal variant introduced in 6. We consider a training run to be a continuous 24-hour trace of the aggregate of Equation (7), cut into consecutive 300 s windows and scored by the tracker. The run's cadence and background are held fixed for the whole day, so the windows of one run are not independent draws. A window counts as detected at false-alarm rate 10^{-2}, as in Section 6. We use an anytime-valid test (Ramdas et al., 2023) to convert a sequence of Viterbi results on a sequence of windows into a decision about the run. Each window's score becomes a p-value against the null (Vovk et al., 2005), each p-value becomes an e-value $e = 1/(2\sqrt{p})$, and the test raises an alarm when the running product of e-values first exceeds $1/\alpha$ [1]. We set $\alpha = 10^{-3}$. Across 2000 null runs the empirical false-alarm rate after a day is 1.5×10^{-3} (95% interval 0.5–4.4×10^{-3}), consistent with α. We also tested a simpler counting rule, which flags a run when the number of windows detected by a fixed time would be improbable for a null facility. It also keeps its false-alarm rate (two false alarms in 2000 null runs after a day), but the observation time must be fixed in advance, and it flags fewer runs (Figure 8b).

Results are presented in Figure 8. Under the work variation strategy at $\sigma_G = 0.7$, runs with $f_0 \lesssim 0.8$ Hz are detected in almost every window, and those above about 1.1 Hz in 8–20%. Even the most difficult of these runs is detected at several times the null's 1% rate, so the test catches it eventually given sufficient observation time (Figure 8b). It flags 68% of runs within an hour, 95% within three hours, and every run within six. For the lognormal work variation of Section 6, runs with f_0 between about 0.9 and 1.3 Hz are detected in only 1–7% of windows, and 15 of 200 runs fall to 2% or below, close to the null's rate. The test flags 44% of these runs within an hour, 75% within six hours, and 87% after a day. The remaining 13% survive a day of observation, and the runs closest to the null's rate would survive much longer. We conclude that a concealing operator can therefore hide a run, but only with the lognormal strategy, only at some cadences, and at the 93% learning cost measured in Section 6.

[1] By Ville's inequality (Ville, 1939), the false-alarm rate stays below α however long the verifier watches and whenever it stops.

7.2. Confounds

A detection by the Viterbi method establishes that the power trace carries a persistent, training-shaped periodic schedule. However it does not establish that the computation is actually training. Ordinary operations could lead to similar periodicities that trip the detector, e.g. a non-ML kernel loop with a training-shaped cycle, a periodic inference request generator, or a controller limit cycle, see Appendix E. Instrumental lines that arise from the hardware itself are also possible. For example, on measured single-A100 traces the spectrum contains a 0.31–0.45 Hz component due to the card's own power-management loop, that appears whenever the load varies in time but not at idle or under constant load [2] Instrumental lines don't help a concealing operator hide training, but it could lead to otherwise compliant operators running inference being erroneously flagged. Gravitational-wave searches face the same problem, and discard candidates that coincide with a catalogue of known instrumental lines (Covas et al., 2018). A verifier could keep a similar list for each facility, with two differences. Building it needs access to characterise the facility's hardware, and an operator who knows the list can move its cadence into a listed band.

7.3. Limitations

The primary limitation to this work is the use of synthetic data generated from a model of data centre power draw, Section 2. The training part of the model, Section 2.1, is grounded in data. Ko & Zhu (2025) take its parameters from published characterisations of datacentre and GPU power: an oscillation near 1 Hz, compute phases that fill 55–80% of each iteration, and a ∼30% drop in draw during synchronisation. They report that the profiles it generates agree qualitatively with public power traces and with the experience of datacentre engineers. They could not test it quantitatively, because facility-scale measurements are proprietary. The inference part of the model, Section 2.3 has weaker support. Ko & Zhu (2025) give no inference model, so the one in Section 2.3 is our own. It follows the published behaviour of serving systems, but we have not checked it against measured traces. The natural extension of this work is to replace the synthetic model with empirical multi-

[2] This line need not grow in step with training at a facility meter. The controllers of N independent cards should add incoherently, growing roughly as $\sqrt{N}$, while a synchronous training line grows as N, so the controller line should fall relative to training at cluster scale.

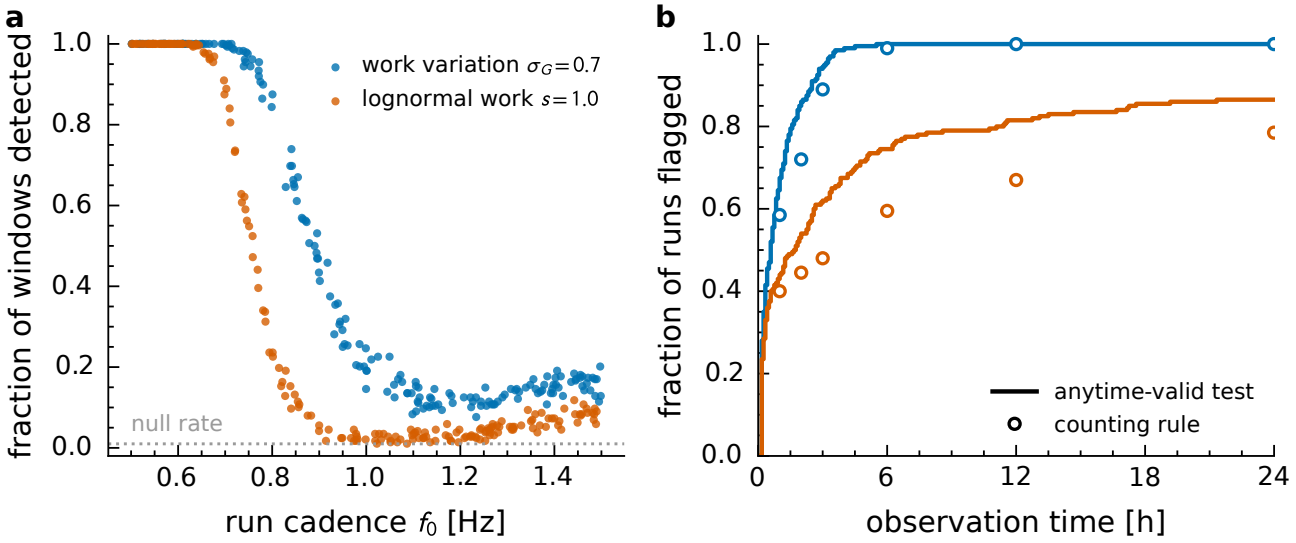


Figure 8. Detection of 200 runs under each of two work-variation strategies, $\sigma_G = 0.7$ (blue) and lognormal (orange). Each run is one continuous 24-hour trace of the aggregate, scored in 300 s windows. **a**, fraction of each run's windows detected at false-alarm rate 10^{-2}, against the run's cadence. The dotted line is the null's 1% rate. **b**, fraction of runs flagged against observation time, at a run-level false-alarm rate of 10^{-3}. Lines are the anytime-valid test, which may stop at any time; open circles are the counting rule, a separate fixed-time test at each marked time. Honest training is flagged in its first window and is not shown. Across 2000 null runs the anytime-valid test raises 3 false alarms in a day.

GPU training observed by an external meter. That study is planned but requires hardware and an experimental setup that are not currently available. Measurements from a single A100, read through NVML, are possible. However they are a poor stand-in for what a facility meter sees, for two reasons. First, one card has no communication phase. On a single saturated A100 the iteration cadence is present but shallow: it modulates a ∼370 W draw by only a few watts (∼0.4% of board power); the deep compute–communication contrast that makes the cadence legible comes from synchronisation across many accelerators, so a single card is the case in which it is weakest. Second, NVML reports one card's power from its own sensors, while a facility meter sees the aggregate of many cards through the power-delivery path. For the quantity a facility meter sees, the scenario model of Ko & Zhu (2025) is therefore the better guide. We use the A100 measurements only where a single card is a fair proxy, e.g. the throughput cost of the concealing strategies (Section 6).

There are a series of additional limitations in our approach. (i) The observation map of Equation (10) is a linear time-invariant first approximation. (ii) The tracker's threshold is set from simulated traces of our modelled null, so its false-alarm rate is guaranteed only if a real facility's null resembles that model. A deployed verifier would instead calibrate on the facility's own training-free traces or on per-trace surrogates (Section 4.2). (iii) Our estimate of the learning efficiency cost in Section 6 was only measured at a small scale with a 30M-parameter model, one dataset, and a limited search over concealing schedules . Whether it holds at frontier scale, and against strategies we did not test, is an open question. (iv) The run-level results of Section 7.1 cover a single day of observation, and the anytime-valid guarantee there assumes the window p-values are independent under the null. The null runs support this for our modelled facility, but a real facility's false alarms could be more correlated. (v) We target synchronous, iteration-structured training whose cadence lies in the searched band; a run that moves its cadence outside the band or abandons synchronous iteration is outside the population we test. (vi) The declared workload need not be pure inference; a facility may be permitted fine-tuning or training below a threshold, whose lines our detector would also find, and telling which run a line belongs to and how large it is needs the integrated-energy channel we exclude. (vii) Our claims on the robustness of the Viterbi method to concealing strategies and cost budgets is limited to those strategies we considered - there may be alternative strategies not considered here that the operator could employ. One important example is energy storage between the accelerators and the meter. An energy store sized to the cadence swing could discharge during each compute phase and recharge during each communication phase, so that the meter sees a flat load. This costs the operator no throughput, but they would incur additional costs such as purchasing power electronics sized for the full swing, maintenance of a store that wears from charging and discharging once per iteration, and energy lost on every cycle. Power smoothing of this kind is a grid mitigation for training load in its own

right (Ko & Zhu, 2025), so it may be installed for reasons unrelated to concealment. The verifier can respond by moving the meter, or adding a second one, between the store and the accelerators, where the load is not yet smoothed. A meter on the store itself also works: to flatten the load, the store must charge and discharge at the training cadence, so its own power flow carries the line. Both meters stay off-chip if the store sits at the facility or row level. If it sits inside the racks, the meter must move closer to the chips than we assume.

8. Conclusion

We have shown that a passive, off-chip power meter can detect synchronous training hidden in a declared inference workload. A Viterbi detector finds the iteration cadence of training, and is robust to many different concealing strategies. Only varying the real work per iteration weakens the detector. Over a long training run, an anytime-valid test catches most attacked runs within a day. Hiding training is expensive: the strongest attack we found evades detection in about one run in seven after a day, but costs the operator 93% of its learning efficiency.

References

Baker, M., Kulp, G., Marks, O., Brundage, M., and Heim, L. Verifying international agreements on AI: Six layers of verification for rules on large-scale AI development and deployment, July 2025. URL https://arxiv.org/abs/2507.15916.

Bayley, J., Messenger, C., and Woan, G. Generalized application of the Viterbi algorithm to searches for continuous gravitational-wave signals. *Physical Review D*, 100: 023006, 2019. doi: 10.1103/PhysRevD.100.023006.

Covas, P. B., Effler, A., Goetz, E., Meyers, P. M., Neunzert, A., et al. Identification and mitigation of narrow spectral artifacts that degrade searches for persistent gravitational waves in the first two observing runs of Advanced LIGO. *Physical Review D*, 97:082002, 2018. doi: 10.1103/PhysRevD.97.082002.

Djurović, I. and Stanković, L. Viterbi algorithm for chirp-rate and instantaneous frequency estimation. *Signal Processing*, 91(5):1308–1314, 2011. doi: 10.1016/j.sigpro.2010.10.007.

Gargiulo, S. and Kulp, G. Workload identification with physical side channels for AI governance, 2026. URL https://arxiv.org/abs/2609.00309.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., et al. The Llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Heim, L. and Koessler, L. Training compute thresholds: Features and functions in AI regulation, 2024. URL https://arxiv.org/abs/2405.10799.

Jain, R., Tran, B., Chen, K., Sinclair, M. D., and Venkataraman, S. PAL: A variability-aware policy for scheduling ML workloads in GPU clusters. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–18. IEEE, 2024. doi: 10.1109/SC41406.2024.00032.

Ko, M.-S. and Zhu, H. Wide-area power system oscillations from large-scale AI workloads, August 2025. URL https://arxiv.org/abs/2508.16457.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165.

Lin, J., Jiang, Z., Song, Z., Zhao, S., Yu, M., Wang, Z., Wang, C., Shi, Z., Shi, X., Jia, W., Liu, Z., Wang, S., Lin, H., Liu, X., Panda, A., and Li, J. Understanding stragglers in large model training using what-if analysis. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025. URL https://arxiv.org/abs/2505.05713.

McCandlish, S., Kaplan, J., Amodei, D., and OpenAI Dota Team. An empirical model of large-batch training, 2018. URL https://arxiv.org/abs/1812.06162.

Patel, P., Choukse, E., Zhang, C., Goiri, Í., Warrier, B., Mahalingam, N., and Bianchini, R. POLCA: Power oversubscription in LLM cloud providers, 2023. URL https://arxiv.org/abs/2308.12908.

Patel, P., Choukse, E., Zhang, C., Goiri, Í., Warrier, B., Mahalingam, N., and Bianchini, R. Characterizing power management opportunities for LLMs in the cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*, pp. 207–222. ACM, 2024a. doi: 10.1145/3620666.3651329.

Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., and Bianchini, R. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 118–132, 2024b. doi: 10.1109/ISCA59077.2024.00019.

Penedo, G., Kydlíček, H., Ben Allal, L., Lozhkov, A., Mitchell, M., Raffel, C., Von Werra, L., and Wolf, T. The FineWeb datasets: Decanting the web for the finest

text data at scale. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2024. URL https://arxiv.org/abs/2406.17557.

Percival, D. B. and Walden, A. T. *Spectral Analysis for Physical Applications: Multitaper and Conventional Univariate Techniques*. Cambridge University Press, 1993. doi: 10.1017/CBO9780511622762.

Rahman, R. and Tajdari, S. Detecting hidden ML training with zero-overhead telemetry, 2026. URL https://arxiv.org/abs/2606.19262.

Ramdas, A., Grünwald, P., Vovk, V., and Shafer, G. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023. doi: 10.1214/23-STS894.

Sastry, G., Heim, L., Belfield, H., Anderljung, M., Brundage, M., Hazell, J., O'Keefe, C., Hadfield, G. K., Ngo, R., Pilz, K., Gor, G., Bluemke, E., Shoker, S., Egan, J., Trager, R. F., Avin, S., Weller, A., Bengio, Y., and Coyle, D. Computing power and the governance of artificial intelligence, 2024. URL https://arxiv.org/abs/2402.08797.

Shavit, Y. What does it take to catch a Chinchilla? Verifying rules on large-scale neural network training via compute monitoring, 2023. URL https://arxiv.org/abs/2303.11341.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q. V., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *5th International Conference on Learning Representations (ICLR 2017)*, 2017. URL https://arxiv.org/abs/1701.06538.

Streit, R. L. and Barrett, R. F. Frequency line tracking using hidden Markov models. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 38(4):586–598, 1990. doi: 10.1109/29.52700.

Suvorova, S., Sun, L., Melatos, A., Moran, W., and Evans, R. J. Hidden Markov model tracking of continuous gravitational waves from a neutron star with wandering spin frequency. *Physical Review D*, 93:123009, 2016. doi: 10.1103/PhysRevD.93.123009.

Thomson, D. J. Spectrum estimation and harmonic analysis. *Proceedings of the IEEE*, 70(9):1055–1096, 1982. doi: 10.1109/PROC.1982.12433.

Ville, J. *Étude critique de la notion de collectif*. Gauthier-Villars, Paris, 1939.

Vovk, V., Gammerman, A., and Shafer, G. *Algorithmic Learning in a Random World*. Springer, 2005.

Wu, T., Wang, W., Yu, Y., Yang, S., Wu, W., Duan, Q., Yang, G., Wang, J., Qu, L., and Zhang, L. FALCON: Pinpointing and mitigating stragglers for large-scale hybrid-parallel training, 2024. URL https://arxiv.org/abs/2410.12588.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538. USENIX Association, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

A. Optimality ceiling

Because we own the generators, we can compare the Viterbi tracker with an optimality ceiling: an approximation to the Neyman–Pearson optimal test between the training and null generators. The optimal test compares how likely a trace is under $\mathcal{H}_1$ and under $\mathcal{H}_0$. Our generators can simulate traces from either class but cannot give the probability of a given trace, so we approximate. We summarise each class by its average power spectrum in the search band, estimated from simulated traces, and ask which spectrum better explains the periodogram of the trace (a Whittle likelihood ratio). The training cadence is unknown, so the training spectrum is averaged over a bank of 61 templates, one per value of f_0, each estimated from 2000 simulated traces with drift levels drawn from the whole σ_f axis. The ceiling therefore knows no more about the drift than the tracker does. Each template fixes the dominant run's cadence over a random background, and the null spectrum is estimated from the null aggregate. The traces that build these spectra are drawn from a stream disjoint from the traces we score, and the ceiling is applied to exactly the populations of Figure 5.

The ceiling is a benchmark, not a usable detector. It knows the exact spectra of both classes, which a verifier monitoring a real facility does not. The tracker assumes only a slowly wandering line, and the ceiling measures how much of the available detection power it recovers.

The ceiling detects every trace at every drift (Figure 5). The training line and the null are therefore separable in principle from a 300 s trace, and the tracker's shortfall, at most 0.04 at $\sigma_f = 1.5$ Hz, is a limit of the detector rather than of the problem. One way to close it is known. Under the hidden-Markov wander model the tracker already assumes, the Viterbi score follows the single best path, while the Neyman–Pearson statistic sums over all frequency paths with the forward algorithm (Streit & Barrett, 1990). We leave that statistic to future work.

The ceiling is optimal only under the Whittle model, which uses the power spectrum alone and so ignores the phase relations between harmonics and any non-Gaussian structure. The true optimum can only be higher. Since the ceiling already detects every trace, this changes nothing here.

The background. The same comparison on a single training run against inference alone, with no background load, gives tracker detection rates of 1.00 up to $\sigma_f = 0.8$ Hz and 0.98 at 1.5 Hz, within 0.02 of the aggregate at every drift. At the default shares the background neither helps nor hinders the tracker.

B. Fixed-frequency baselines

This appendix defines the multitaper F-test of Section 4.3. Taper the mean-removed trace with K discrete prolate spheroidal sequences $v_k[n]$ of time–bandwidth product NW, let $y_k(f)$ be the Fourier transform of $v_k[n]\,x[n]$, and let $U_k = \sum_n v_k[n]$ be the k-th taper's gain at zero frequency. The least-squares amplitude of a line at f and the harmonic F statistic are (Thomson, 1982; Percival & Walden, 1993)

$$\hat{\mu}(f) \;=\; \frac{\sum_k U_k\, y_k(f)}{\sum_k U_k^2}, \qquad F(f) \;=\; (K-1)\,\frac{|\hat{\mu}(f)|^2 \sum_k U_k^2}{\sum_k \bigl|y_k(f) - \hat{\mu}(f)\,U_k\bigr|^2}, \tag{21}$$

the power in the fitted line over the power the tapers see around it. We use $K = 7$ and $NW = 4$, on a zero-padded frequency grid.

At a single fixed frequency, and for a stationary null whose spectrum is smooth near f, $F(f)$ follows an $F(2, 2K-2)$ distribution. The test statistic of Equation (17) is instead the maximum of F over the search band, so its null distribution needs a correction for the number of frequencies searched. The analytic correction does not work. A Bonferroni correction over the number of independent frequency cells sets thresholds that are too low: the realised false-alarm rate exceeds the nominal one by a factor of about 3.6–6 on every null we tried, because F fluctuates on scales finer than a frequency cell. Real power traces would also break the smooth-spectrum assumption. We therefore threshold both fixed tests empirically, as for the tracker (Section 4.2).

C. Meter specification details

This appendix gives two results behind the minimum meter specification of Section 5.2: the notch sweep, and a check that the specification does not depend on the background load.

The notch. Figure 9 shows the tracker's detection rate as a delivery-path notch moves across the training cadence band of Table 1 (0.5–1.5 Hz), with the meter otherwise nominal. The notch has quality factor $Q = 1.5$ (Table 2), so it is broad: about 0.7 Hz wide at 1 Hz. A notch of 50% depth costs nothing at any centre, and one at 0.6 Hz, near the low edge of the band, costs at most 0.02. Deeper notches nearer the band centre cost more. At 0.8 Hz the tracker falls to 0.86 (90% depth) and 0.80 (100%), and at 1.0–1.2 Hz to 0.45–0.63. No notch in the sweep removes detection entirely.

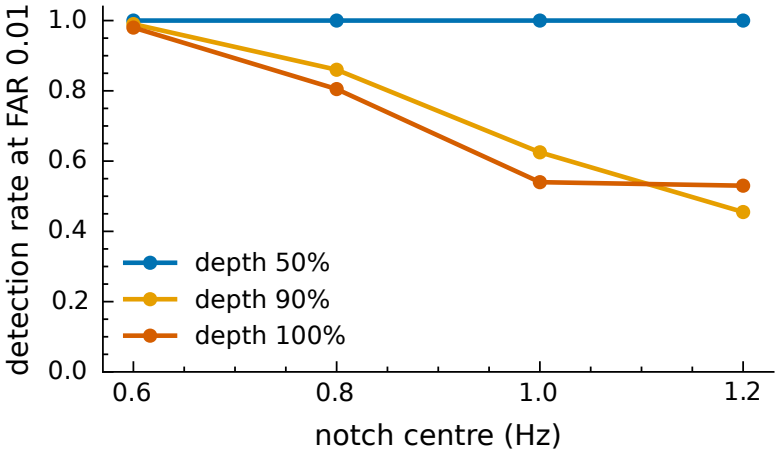


Figure 9. Notch sweep: Viterbi detection rate at empirical false-alarm rate 10^{-2} versus notch centre, one line per notch depth, at 20 Hz sampling with no integration window, the honest aggregate against the null aggregate ($n = 200$ per class). A deep notch near the band centre roughly halves detection; none removes it.

The background. The same sweep on a single training run against inference alone, with no background load, changes the tracker's detection rate by at most 0.09 in any cell and leaves the region where detection holds ($f_s \geq 2$ Hz and $\tau \leq 0.5$ s) unchanged. As for the drift sweep (Appendix A), the background at the default shares does not move the result.

D. Throughput-cost measurements

This appendix records how the sweep of Section 6 was run and how the throughput costs of Figure 7 were measured. The sweep applies each strategy of Table 3 to the dominant training run of the aggregate and scores it against the null aggregate, with $n = 200$ traces per class over 50 cells, every rate read at false-alarm rate 10^{-2}.

The throughput cost c of Equation (18) comes from a campaign on a single A100. We use only its throughput measurements. We do not use the size of the power line it measured, since a single card is a poor guide to what a facility meter sees (Section 7.3), so the costs do not depend on that caveat. Each cost is the mean over eight A100 traces at that setting. Work variation and jitter at level 0.7, and drift at 0.1, 0.4, 0.8 and 1.5 Hz, were measured in a later run on the same hardware. The drift costs are heavy-tailed: at 0.8 and 1.5 Hz the mean exceeds the median by factors of 1.5 and 2. For work variation the campaign records the throughput deficit $d = 1 - r/r_{\text{honest}}$ rather than c. The two are related by $c = d/(1 - d)$ and differ by less than 0.01 percentage points at the measured $|d| < 1\%$, so we report d directly.

E. Non-training loads that score as training

Section 7.2 notes that a load can carry a training-shaped power schedule without training. This appendix tests that with loads whose training status we know, using $n = 200$ traces of 300 s per class.

Setup. We go beyond the single Viterbi score and give the verifier a classifier. Each trace is reduced to eight features computed from the trace alone: six detector scores (multitaper F-test, harmonic comb, spectral matched filter, Viterbi score, and two cyclostationary statistics), plus the stability of the Viterbi path over time and how repeatable the power waveform is from one iteration to the next. Three decision rules read these features: a fixed linear score, a logistic regression, and a random forest. Each is trained to separate training from the inference null of Section 2.3. All three do so perfectly, with area under the ROC curve 1.00, and keep that score when the population is shifted (device headroom, trace length, cadence band, an added controller line, heavy coloured meter noise).

Controls. We then score seven control loads at a false-positive rate of 0.05 (Table 4). Five are not training: a non-ML kernel loop with a training-shaped cycle, a power-controller limit cycle, a periodic inference request generator, a computation of gradients alone, and a decoy that runs the full forward pass, backward pass and optimiser step and then discards the update. Every rule scores all five as training. Two are training: a run diluted by an unrelated co-resident load, which every rule catches, and an asynchronous run whose cadence drifts and slips, which the rules catch in 55–100% of traces.

Table 4. Fraction of each control load scored as training at a false-positive rate of 0.05, for each decision rule. All five non-training loads score as training.

Control load	Training?	Linear	Logistic	Forest
Discarded-update decoy	no	1.00	1.00	1.00
Gradient-only	no	1.00	1.00	1.00
Non-ML kernel loop	no	1.00	1.00	1.00
Controller cycle	no	1.00	1.00	1.00
Periodic inference	no	1.00	1.00	1.00
Co-resident mixture	yes	1.00	1.00	1.00
Asynchronous training	yes	0.72	0.55	1.00

Why no detector can do better. The meter sees only the power schedule. A verifier's decision is some function of the trace, so two loads that produce traces with the same distribution are flagged with the same probability, by any detector. The decoy reproduces the alternation of compute and communication that makes up the training schedule, so it produces the same trace as training without training a model. Amplitude does not help either: the power drawn is the rate of operations times the energy per operation, and the meter sees only the product. A higher sampling rate does not help. A classifier reading a 10 MHz board-level probe separates training from inference by the power signature of the optimiser update (Gargiulo & Kulp, 2026), but the decoy runs that update and only discards its result. A detection therefore shows that a training-shaped schedule is running, not that a model is being trained. For the verifier the practical concern is the benign loads in Table 4, such as the controller cycle and the kernel loop, which could flag a compliant facility; a concealing operator has no reason to run a decoy (Section 3.2).